Design Patterns 1

Programare Orientată pe Obiecte

Proiectarea aplicaţiilor

- Realizarea unui program presupune e*tapa iniţială de proiectare*
- Este posibil ca după implementarea proiectului iniţial să se revină la această etapă
  - Îmbunătăţire – refactorizare
  - Modificare esenţială
- Faza de proiectare
  - Se stabilesc clasele, interfeţele necesare şi relaţiile dintre ele
    - Clase noi specifice aplicaţiei
    - Clase predefinite de uz general
  - Pentru un proiect mai complex: crearea de modele ce reprezintă relaţiile dintre clase şi obiecte, rolul fiecărui obiect etc. - UML (Unified Modeling Language)

Proiectarea aplicaţiilor

- Proiectare ce are în vedere posibilitatea de a fi extins şi adaptat ulterior într-un mod cât mai simplu şi cât mai sigur – *design for change*

- Extindere simplă şi sigură
  - Nu se va recurge la modificarea unor clase existente
  - Se vor defini clase noi care
    - Înlocuiesc clasele din programul existent
    - Se vor adăuga claselor existente

- Posibilitatea reutilizării unor clase din aplicaţie şi în alte aplicaţii

Proiectarea aplicaţiilor

- Clasele si obiectele necesare într-o aplicatie pot rezulta din:
  - Analiza aplicaţiei concrete şi modelarea obiectelor şi acţiunilor din aplicaţie;
  - Analiza altor aplicaţii şi a bibliotecilor standard Java =>
    - Clase cu caracter mai abstract pentru:
      - Grupare de obiecte
      - Intermediere între obiecte
      - Alt rol care nu este evident din descrierea aplicaţiei
  - Cunoaşterea unor soluţii de proiectare deja folosite cu succes în alte aplicaţii – *design patterns*

Proiectarea aplicaţiilor

- Mărirea flexibilităţii în extinderea şi adaptarea unei aplicaţii:
  - prin mărirea numărului de clase si obiecte din aplicaţie!
- Un proiect ce conţine numai clasele rezultate din analiza aplicaţiei poate fi mai compact
  - dar nu este scalabil şi adaptabil la alte cerinţe apărute după realizarea prototipului aplicaţiei!
- Sunt importante:
  - separarea părţilor susceptibile de schimbare de părţile care nu se mai modifică
  - evitarea cuplajelor "strânse" dintre clase

Design Patterns

- Recunoaşterea şi inventarierea unor scheme (şabloane) de proiectare “Design Patterns”:
  - grupuri de clase şi obiecte care cooperează pentru realizarea unor funcţii.
- În cadrul acestor scheme există clase care au un anumit rol în raport cu alte clase şi care au primit un nume ce descrie acest rol;
- Exemple: clase iterator, clase observator, clase “fabrică” de obiecte, clase adaptor ş.a.
- Dincolo de detaliile de implementare se pot identifica clase cu aceleaşi responsabilităţi în diferite aplicaţii.

Design Patterns. Definiţii

- Definitii posibile pentru schemele de proiectare folosite în aplicaţii cu clase:
- Soluţii optime generale şi reutilizabile ale unei probleme comune în design-ul software.
- Descrieri ale soluţiilor sau template-uri ce pot fi aplicate pentru rezolvarea problemei.
- Reguli pentru realizarea anumitor sarcini în proiectarea programelor cu obiecte.
- Abstractizări la un nivel superior claselor, obiectelor sau componentelor.
- Scheme de comunicare (de interacţiune) între obiecte.

Avantaje şi recomandări

- Argumentul principal în favoarea studierii si aplicării schemelor de clase:
  - aplicarea acestor scheme conduce la programe mai usor de modificat.
    - în general prin clase (obiecte) “slab” cuplate, care stiu cât mai putin unele despre altele.

- Principalele recomandări care rezultă din analiza schemelor de proiectare şi a aplicaţiilor:
  - Proiectarea cu interfeţe şi clase abstracte este preferată faţă de proiectarea cu clase concrete
    - permite separarea utilizării de implementare.
  - Este recomandată crearea de clase si obiecte suplimentare, cu rol de intermediari
    - pentru decuplarea unor clase cu roluri diferite.

Clasificare

Se disting trei categorii:

1. Creational patterns: scheme “creaţionale“
   - generează obiectele necesare
   - mecanisme de creare a obiectelor
   - Ex. *Singleton*, *Factory* etc

2. Structural patterns: scheme structurale
   - definesc relaţii între entităţi
   - grupează mai multe obiecte în structuri mai mari.
   - Decorator, Adapter, Composite, Proxy etc.

3. Behavioral patterns: scheme de interacţiune
   - definesc comunicarea între entităţi
   - *Iterator*, *Visitor*, *Observer*, *Command*, Mediator etc.

Design Pattern
Singleton

Programare Orientată pe Obiecte

Creational Patterns

- mecanisme de creare a obiectelor
  - **Singleton**
  - **Factory**

Singleton Pattern

- utilizat pentru a restrictiona numarul de instantieri ale unei clase la un singur obiect
- o metodă de a folosi o singură instanţă a unui obiect în aplicaţie.

Utilizări

- subansamblu al altor pattern-uri:
  - împreună cu pattern-urile Abstract Factory, Builder, Prototype etc.
- obiectele care reprezintă stări
- în locul variabilelor globale.
  - Preferat variabilelor globale deoarece, printre altele, nu poluează namespace-ul global cu variabile care nu sunt necesare.

Implementare

- Implementarea unei metode statice, publice, ce permite crearea unei noi instanţe a clasei dacă aceasta nu exista deja.
- Daca instanţa există deja, atunci întoarce o referinţă către acel obiect.
- Pentru a asigura o singură instanţiere a clasei, constructorul trebuie să fie *private*
- clasa respectivă va fi instanţiată **lazy** (*lazy instantiation*), utilizând memoria doar în momentul în care acest lucru este necesar deoarece instanţa se creează atunci când se apelează metoda ce întoarce instanţa
- Avantaj faţă de clasele non-singleton, pentru care se face *eager instantiation* - se alocă memorie încă de la început, chiar dacă instanţa nu va fi folosită

Implementare

- putem implementa o componentă de acest tip în mai multe feluri, inclusiv folosind enum-uri în loc de clase.
- trebuie avut în vedere contextul în care îl folosim, astfel încât să alegem o soluţie care să funcţioneze corect în toate situaţiile ce pot apărea în aplicaţie
  - unele implementări au probleme atunci când sunt accesate din mai multe thread-uri sau când trebuie serializate

Utilizare:

- dacă avem nevoie să menţinem o referinţă către un obiect în mai multe clase, putem să facem clasa Singleton şi să obţinem acel obiect în fiecare componentă în care avem nevoie de el
- cod mai curat şi cu un flow mai uşor de urmărit, să îl instanţiem doar într-un singur loc şi să îl transmitem ca argument.

Implementare - variantă

```
public class Singleton {

    private static Singleton instance = new Singleton();

    private Singleton() { }

    public static Singleton getInstance() {
            return instance;
    }

}
```

- Instanţa singleton este *private*
- Constructorul definit este constructorul privat pentru a împiedica definirea constructorului default

Implementare - variantă

```
public class Singleton {
    private static final Singleton singleton = new Singleton( );

    // Constructor privat – alte clase nu pot instanţia
    private Singleton(){ }

    // metoda statică ce intoarce instanta
    public static Singleton getInstance( ) {
            return singleton; }

    // Alte metode protejate de singleton-ness
    protected void demoMethod( ) {
            System.out.println("demoMethod for singleton"); }
}
class SingletonDemo {
    public static void main(String[] args) {
            Singleton tmp = Singleton.getInstance( );
            tmp.demoMethod( ); }
}
```

Implementare - variantă

```
public class ClassicSingleton {

    private static ClassicSingleton instance = null;

    private ClassicSingleton() {
       // doar pentru a nu permite instanţierea
    }

    public static ClassicSingleton getInstance() {
        If (instance == null) {
          instance = new ClassicSingleton();
        }
        return instance;
    }
}
```

Observaţii

- lazy instantiation – pentru crearea singleton-ului;
- instanţa este iniţial nulă
- instanţa singleton nu este creată până când metoda getInstance() nu este apelată pentru prima oară.
- această tehnică asigură faptul că instanţa este creată doar atunci când este nevoie de ea.

Tipuri de date enumerare

```
enum

public class CuloriSemafor {
      public static final int ROSU = -1;
      public static final int GALBEN = 0;
      public static final int VERDE = 1;
}

...
// Exemplu de utilizare
if (semafor.culoare == CuloriSemafor.ROSU)
semafor.culoare = CuloriSemafor.GALBEN;

// Doar de la versiunea 1.5 !

public enum CuloriSemafor { ROSU, GALBEN, VERDE };
// Utilizarea structurii se face la fel
```

Implementare - enum

- Enum furnizează suport pentru thread safety şi este garantată o singură instanţă
- Modalitate bună de a avea un singleton cu un minim de efort.

```
public enum EnumSingleton {
    INSTANCE;
    public void someMethod(String param) {
        // some class member
    }
}
```

Aplicabilitate

- utilizat des în situaţii în care avem obiecte care trebuie accesate din mai multe locuri ale aplicaţiei:
- obiecte de tip logger
- obiecte care reprezintă resurse partajate (conexiuni, sockets etc.)
- obiecte ce conţin configuraţii pentru aplicaţie
- pentru obiecte de tip Factory.

Exemple din API-ul Java:

- java.lang.Runtime
- java.awt.Toolkit

Design Pattern

Factory

Programare Orientată pe Obiecte

Factory

- Oferă o soluţie legată de crearea obiectelor =>face parte din categoria *Creational Patterns*
- Folosite pentru obiecte care generează instanţe de clase înrudite (implementează aceeaşi interfaţă, moştenesc aceeaşi clasă abstractă).
- Acestea sunt utilizate atunci când dorim să izolăm obiectul care are nevoie de o instanţă de un anumit tip de crearea efectivă a acesteia.
- Clasa care va folosi instanţa nici nu are nevoie să specifice exact clasa obiectului ce urmează a fi creat, deci nu trebuie să cunoască toate implementările acelui tip, ci doar ce caracteristici trebuie să aibă obiectul creat.
- Situaţia cea mai întâlnită: trebuie instanţiate mai multe clase care implementează o anumită interfaţă sau extind o altă clasă (eventual abstractă).
- Clasa care foloseşte aceste subclase nu trebuie să “ştie” tipul lor concret ci doar pe al părintelui.

Implementare

```
public class ProductFactory{

    public Product createProduct(String ProductID){
      switch (id){
          case ID1: return new OneProduct();
          case ID2: return new AnotherProduct();
          ... // la fel pt alte produse
          default: return null;
       }
    }
    ...
}
```

Exemplu

```
abstract class User{
    public abstract String getRole();
}

class Admin extends User {
    public String getRole() {
        return “ADMIN”;
    }
}

class Student extends User {
    public String getRole() {
        return “STUDENT”;
    }
}
```

Exemplu

```
class UserFactory {
    public enum UserType {
        Adm,
        Stud
    }
    public static User createUser(UserType userType) {
        switch (userType) {
            case Adm:
                return new Admin();
            case Stud:
                return new Student();
      }
        throw new IllegalArgumentException("The user type " +
     userType + " is not recognized.");
    }
}
```

Exemplu

```
class UserCreation{
    /*
     * Create all available users
     */
    public static void main (String args[]) {
        for (UserFactory.UserType userType:
                                    UserFactory.UserType.values()) {
                System.out.println(“Role of " + userType + " is " +
     UserFactory.createUser(userType).getRole());
        }
    }
}
```

Output:

Role of Adm is ADMIN

Role of Stud is STUDENT

Utilizare Factory şi Singleton

- *De ce am avea nevoie de această combinaţie?*
- De obicei avem nevoie ca o clasă *factory* să fie utilizată din mai multe componente ale aplicaţiei
- Este suficient să avem o singură instanţă a factory-ului şi să o folosim pe aceasta.
- Folosind pattern-ul Singleton putem face clasa *factory* un *singleton*

=> din mai multe clase putem obţine instanţa acesteia şi cu ajutorul ei să fie create instanţe ale unor resurse ale căror tip nu este cunoscut codului ce le utilizează.

Exemplu - Java Abstract Window Toolkit (AWT)

- *java.awt.Toolkit*
- clasă abstractă ce face legatura dintre componentele AWT şi implementarile native din toolkit.
- are o metoda factory *Toolkit.getDefaultToolkit()* ce întoarce subclasa de Toolkit specifică platformei
- obiectul Toolkit este un Singleton deoarece AWT are nevoie de un singur obiect pentru a efectua legaturile şi deoarece un astfel de obiect este destul de costisitor de creat.
- Metodele trebuie implementate în interiorul obiectului şi nu pot fi declarate statice deoarece implementarea specifică nu este cunoscută de componentele independente de platformă.

Design Pattern
Visitor

Programare Orientată pe Obiecte

Visitor pattern

- *Behavioral pattern* - comportamental

- Modalitate de a *separa un algoritm* de *structura* pe care acesta operează.

Putem adăuga noi:

- posibilităţi de prelucrare a structurii, fără să o modificăm
- funcţii care realizează prelucrări asupra unei familii de clase, fără a modifica efectiv structura claselor

Structura:

- Element (sau *Observable*) şi Visitor
- Element: are o metodă *accept* care poate primi ca argument un *Visitor*
  - Metoda *accept* apelează metoda *visit* din *Visitor; Element-ul* se transmite pe sine însuşi ca argument al metodei *visit*

Exemplu

- Algoritm pentru parcurgerea elementelor unui graf
- Mai multe operaţii ce pot fi realizate în timpul acestei parcurgeri prin furnizarea a diferite tipuri de vizitatori care să interacţioneze cu elementele grafului
- Are la bază tipul dinamic atât al elementelor grafului cât şi al vizitatorilor – double dispatch!
- Double dispatch - Metoda apelată este determinată la *runtime* de doi factori!

Double Dispatch

Atunci când metoda *accept* e apelată în program, implementarea ei este determinată pe baza:

- Tipului dinamic (la execuţie) al *Element-ului* şi
- Tipului static al *Visitor-ului*.

Atunci când metoda asociată *visit* e apelată în program, implementarea ei este determinată pe baza:

- Tipului dinamic al *Visitor-ului* şi
- Tipului static al *Element-ului* aşa cum este el cunoscut din implementarea metodei *accept*, care înseamnă de fapt tipul dinamic al *Element-ului*

Prin urmare, implementarea metodei *visit* este determinată pe baza:

- Tipului dinamic al *Element-ului şi*
- Tipului dinamic al *Visitor-ului*

Exemplul 1

- Afişarea conţinutului unui arbore de noduri, care în acest caz descrie componentele unei maşini.
- În loc să scriem metoda "print" pentru fiecare subclasă (Wheel, Engine, Body şi Car), avem o singură clasă (CarElementPrintVisitor) ce realizează acţiunea de afişare dorită.
- Deoarece diferitele subclase necesită acţiuni uşor diferite pentru a afişa corect, CarElementDoVisitor determină acţiunile pe baza clasei argumentului primit de către el.

Implementare

```
interface ICarElementVisitor {
    void visit(Wheel wheel);
    void visit(Engine engine);
    void visit(Body body);
    void visit(Car car);
}

interface ICarElement {
    void accept(ICarElementVisitor visitor);
}

class Wheel implements ICarElement {
    private String name;
    public Wheel(String name) {
        this.name = name;
    }
    public String getName() {
        return this.name;
    }
```

Implementare

```
    public void accept(ICarElementVisitor visitor) {
        /*
         * accept(ICarElementVisitor) din Wheel implementeaza
         * accept(ICarElementVisitor) din ICarElement, astfel incat
         * apelul lui accept se face prin dynamic binding (la execuţie).
         * Acesta poate fi considerat “the first dispatch”.
         * Decizia de a apela  visit(Wheel) (si nu visit(Engine) etc.)
         * poate fi luată la compilare deoarece 'this' este cunoscut în
         * momentul compilării ca fiind un Wheel. Pe lângă aceasta, fiecare
         * implementare  a lui ICarElementVisitor implementează
         * visit(Wheel), care este o altă decizie ce este luată în momentul
          * execuţiei. Acesta poate fi considerat “the second dispatch”.
         */
        visitor.visit ( this );
    }
}
class Engine implements ICarElement {
    public void accept(ICarElementVisitor visitor) {
        visitor.visit ( this );
    }
}
```

Implementare

```
class Body implements ICarElement {
    public void accept(ICarElementVisitor visitor) {
        visitor.visit ( this );
    }
}
 class Car implements ICarElement {
    ICarElement[ ] elements;
    public Car() {
         elements = new ICarElement[] { new Wheel("front left"),
            new Wheel("front right"), new Wheel("back left") ,
            new Wheel("back right"), new Body(), new Engine() };
    }
    public void accept(ICarElementVisitor visitor) {
        for( ICarElement elem : elements) {
             elem.accept (visitor);
        }
        visitor.visit ( this );
    }
}
```

Implementare

```
class CarElementPrintVisitor implements ICarElementVisitor {

     public void visit (Wheel wheel) {
        System.out.println("Visiting " + wheel.getName() + " wheel");
    }

    public void visit(Engine engine) {
        System.out.println("Visiting engine");
    }

    public void visit(Body body) {
        System.out.println("Visiting body");
    }

    public void visit(Car car) {
        System.out.println("Visiting car");
    }
}
```

Implementare

```
class CarElementDoVisitor implements ICarElementVisitor {
    public void visit(Wheel wheel) {
        System.out.println("Kicking my " + wheel.getName() + " wheel");
    }
    public void visit(Engine engine) {
        System.out.println("Starting my engine");
    }
    public void visit(Body body) {
        System.out.println("Moving my body");
    }
    public void visit(Car car) {
        System.out.println("Starting my car");
    }
}
public class VisitorDemo {
    public static void main(String[] args) {
        ICarElement car = new Car();
        car.accept(new CarElementPrintVisitor());
        car.accept(new CarElementDoVisitor());
    }
}
```

Output

```
Visiting front left wheel
Visiting front right wheel
Visiting back left wheel
Visiting back right wheel
Visiting body
Visiting engine
Visiting car
Kicking my front left wheel
Kicking my front right wheel
Kicking my back left wheel
Kicking my back right wheel
Moving my body
Starting my engine
Starting my car
```

Observaţie

- O abordare mai flexibilă a acestui pattern este de a crea o *clasă wrapper* care implementează interfaţa care defineşte metoda *accept*.
- Această clasă conţine o referinţă la ICarElement care poate fi iniţializată prin constructor.
- Această abordare evită necesitatea ca fiecare element să implementeze o interfaţă.

```
class Wrapper implements ICarElement {
     ICarElement el;
    public Wrapper(ICarElement el){
          this.el=el;}
    public void accept(ICarElementVisitor visitor) { }
}
```

Exemplul 2 - Implementare

```
interface Visitor {
        public void visit(Student e);
        public void visit(Profesor b);
}
interface Visitable {
        public void accept(Visitor v);
}
class Student implements Visitable {
        ...
        public void accept(Visitor v) {
                v.visit(this);
        }
}
class Profesor implements Visitable {
        ...
        public void accept(Visitor v) {
                v.visit(this);
        }
}
```

Implementare

```
public class Test {
        public static void main(String[] args) {
                ...
                Visitor v = new ConcreteVisitor();
                for (Visitable e : persoane) // colectie de Persoane
                        e.accept ( v );
        }
}
```

- vizitator ce afişează datele fiecărei persoane (Student sau Profesor):

```
public class ConcreteVisitor implements Visitor {
        public void visit(Student s) {
                System.out.println(s.nume() + " " + s.grupa());
        }
        public void visit(Profesor p) {
                System.out.println(p.nume() + " " +  + p.materia());
        }
}
```